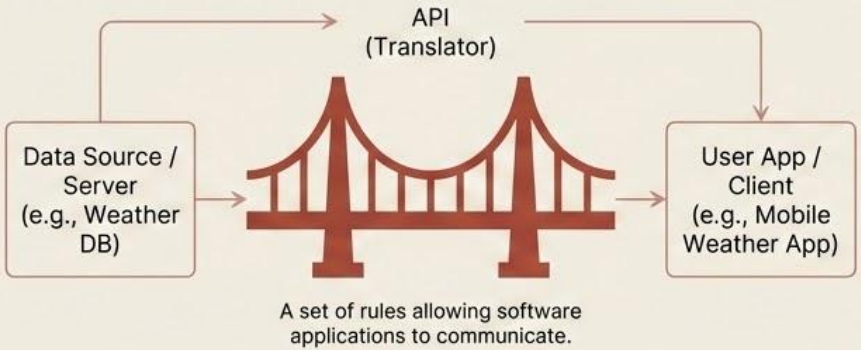
API: The Connector of the Digital World



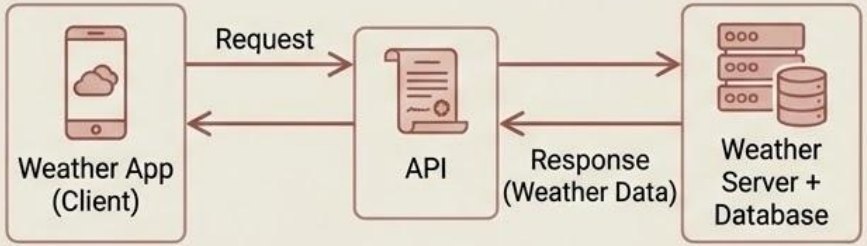
Key Uses

- Data Integration: Connecting disparate systems and silos.
- Feature Extension: Adding functionality (e.g., maps, payments) easily.
- Automated Workflows: Triggering actions across platforms.
- Cloud Services Access: Leveraging external computational power.

What is an API?

An API is a contract that allows one software system to talk to another without knowing how it works internally.

Brutally Simple Example



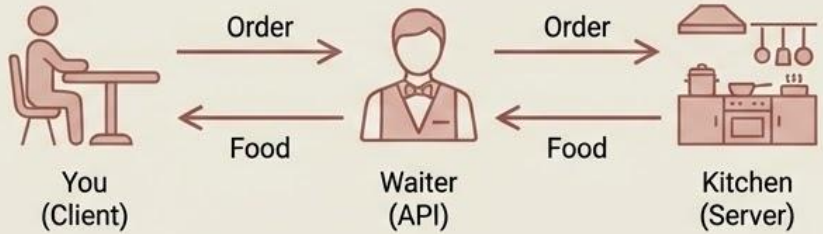
The API defines:

- What request you must send
- What format the response comes in
- What data you're allowed to access

You don't see the server code or database.

You just use the API.

Real-World Analogy



- You = Client
- Kitchen = Server
- Waiter = API

You don't enter the kitchen or cook. You place an order through the waiter. That waiter is the API.

API Data Formats: JSON & XML

APIs use agreed-upon formats to structure and exchange data between systems. The two most common are JSON and XML.

🔥 JSON (JavaScript Object Notation)

```
{
  "user": {
    "id": 123,
    "name": "John Doe",
    "email": "john@example.com",
    "active": true
  }
}
```

- ✔ Lightweight & compact
- ✔ Easy to read & write
- ✔ Native to JavaScript
- ✔ Widely used in modern web/mobile APIs

🔥 XML (eXtensible Markup Language)

```
<user>
  <id>123</id>
  <name>John Doe</name>
  <email>john@example.com</email>
  <active>true</active>
</user>
```

- ✔ Self-descriptive & structured
- ✔ More verbose than JSON
- ✔ Supports complex data types & schemas
- ✔ Common in enterprise & legacy systems

Choice depends on factors like simplicity, performance, and required structure.

🔥 Markup Languages: Structuring Data & Content

A system for annotating a document in a way that is syntactically distinguishable from the text.

🔥 HTML (HyperText Markup Language)

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h1>This is a Heading</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

- ✔ Standard for web pages
- ✔ Defines structure & content
- ✔ Interpreted by browsers
- ✔ Uses predefined tags

🔥 XML & Markdown

```
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

```
# Heading 1
## Heading 2
 * List item 1
 * List item 2
**Bold text**
```

- ✔ XML: Versatile data transport
- ✔ XML: Custom tags allowed
- ✔ Markdown: Simplified formatting
- ✔ Markdown: Easy-to-read plain text

Markup languages provide instructions for how data should be structured, displayed, or processed.

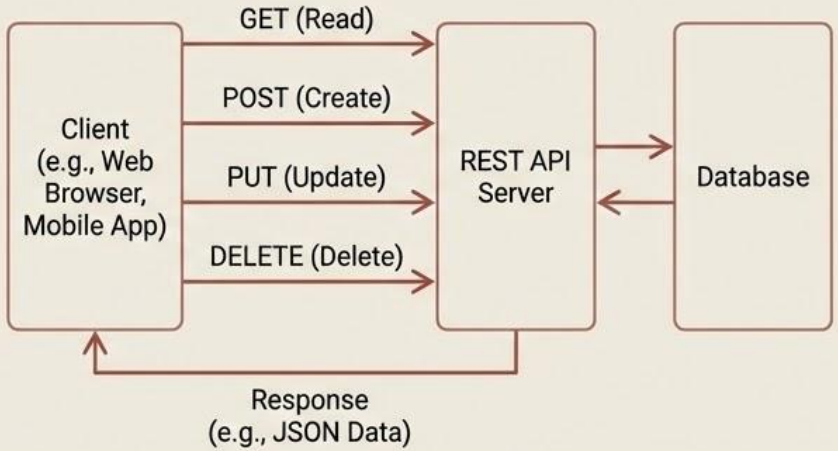
What is REST API?

REST (Representational State Transfer) is an architectural style for building scalable web services. It uses standard HTTP methods and treats data as resources, making interactions simple and efficient.

Key Principles

- **Client-Server**: Separation of user interface and data storage concerns.
- **Stateless**: Each request from client to server must contain all necessary information.
- **Cacheable**: Responses can be stored to improve performance.
- **Uniform Interface**: Consistent way to interact with resources (e.g., using standard HTTP methods).
- **Layered System**: Client doesn't know if it's connected directly to the end server.

How REST API Works



What is REST API?

REST = Representational State Transfer

A REST API is a way for a client (frontend, mobile app, etc.) to communicate with a server using HTTP methods in a structured, predictable way. If you're building React + Node and you don't understand REST properly, you're just copy-pasting tutorials.

Core Idea (No Fluff)


Resources


HTTP Methods


Stateless communication

Standard URL structure

1️⃣ Resource-Based Design

Everything is treated as a resource.
Example: Users → /users, Products → /products, Orders → /orders

❌ Bad beginners write

```
/getUsers
/createUser
/deleteUser
```

That's trash design.

✅ Good REST design

```
GET /users
POST /users
DELETE /users/5
PUT /users/5
```

Same URL. Different HTTP method.

What is a RESTful API?

A RESTful API is an API that correctly follows REST principles.

This is where most people are weak. Many say “REST API” but build garbage endpoints like:

❌ `/getAllUsers`
`/deleteUserById` That’s NOT RESTful.
`/createNewUserNow`

✅ A RESTful API:

- ✅ Uses nouns (users, products, orders)
- ✅ Uses proper HTTP methods
- ✅ Is stateless
- ✅ Returns proper HTTP status codes
- ✅ Uses standard structure

So: **REST** = architectural style
REST API = API using REST
RESTful API = API that follows REST rules properly.

🔥 Example: E-Commerce Web App

Imagine an online store. It uses HTTP, JSON, Stateless requests, Express backend.
So technically, it's using REST concepts.

The "Not RESTful" Approach

❌
```
POST /getAllProducts
POST /createNewProduct
POST /deleteProductById
POST /updateProductData
```

What's wrong?

- Using verbs in URL
- Using POST for everything
- Not using resource-based structure
- Ignoring proper HTTP methods

This is:

✔ A web API ✔ Stateless
✔ Using HTTP ✔ JSON-based

So yes — it is REST-style. But it is NOT RESTful.

🔥 Proper RESTful Version

✔
```
GET    /products
GET    /products/10
POST   /products
PUT    /products/10
DELETE /products/10
```

Now:

- URLs use nouns
- HTTP methods are meaningful
- Resource-based structure
- Clean and predictable

This is RESTful.

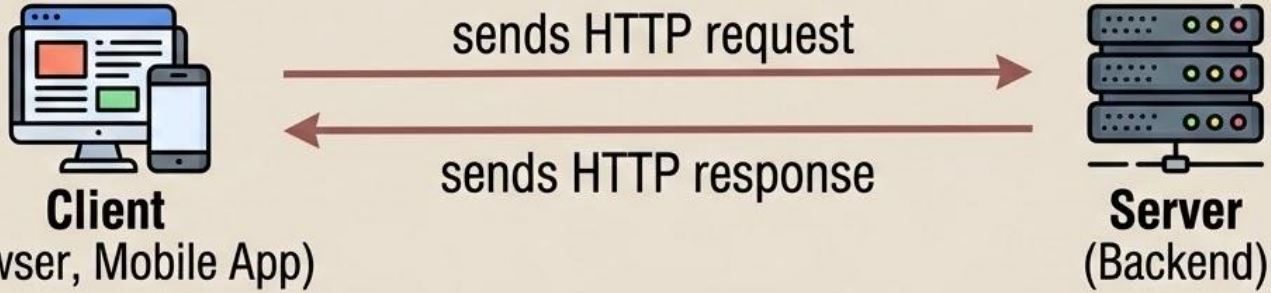
What is a Protocol? What is HTTP?

🔥 What is a Protocol?

A set of rules and conventions for communication between network devices. Like grammar for computers, ensuring they understand each other.

🔥 What is HTTP? (HyperText Transfer Protocol)

The foundation of data communication for the World Wide Web. It defines how messages are formatted and transmitted.

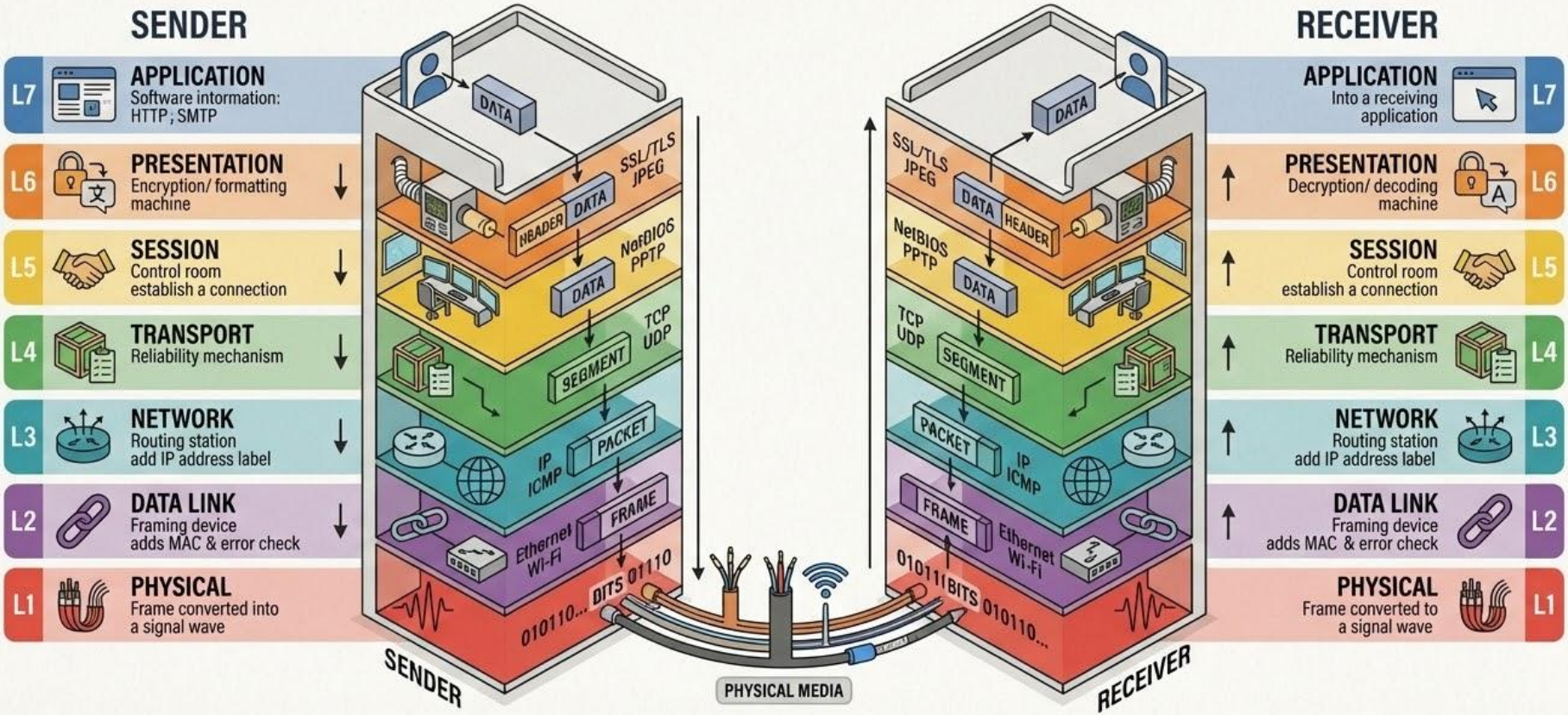


When you open a website: Browser → HTTP Request, Server → HTTP Response. That's it.

Without HTTP, your browser and server don't know how to talk.

THE OSI MODEL: A FRAMEWORK FOR NETWORK COMMUNICATION

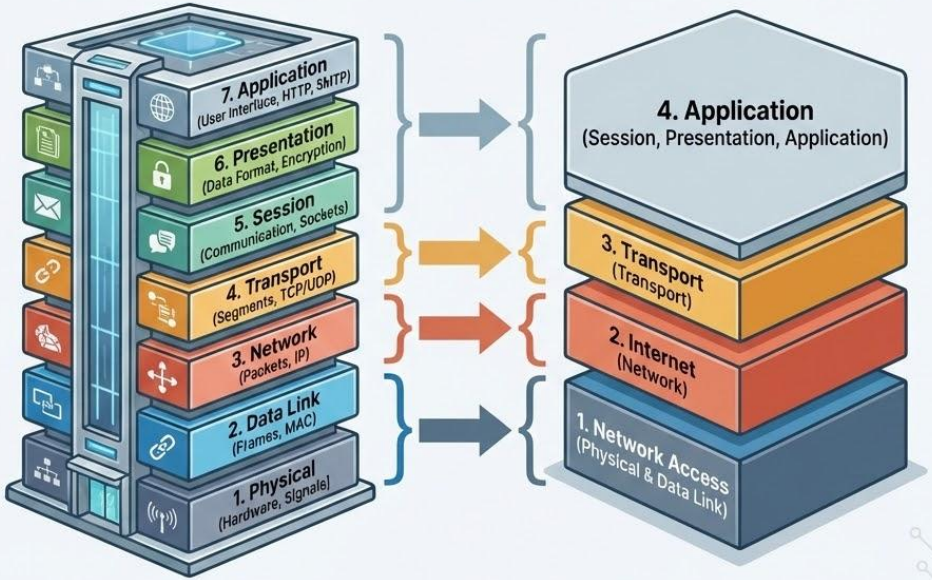
7 Layers of Data Flow, Encapsulation, and Protocols



OSI vs. Reality: Breaking the Network Stack

OSI Model
(Conceptual - 7 Layers)

TCP/IP Model
(Real-World - 4 Layers)



Used for Learning & Reference

Powers the Internet

OSI is a conceptual framework. The Real Internet runs on the streamlined TCP/IP stack.

TRACKING A WEB REQUEST: A JOURNEY THROUGH NETWORK LAYERS

BROWSER SIDE (DOWNWARD FLOW)

Layer 7 — Application Layer (HTTP)

"What do I want?"

```
GET /users HTTP/1.1
Host: example.com
```

Decides Method (GET), Path (/users), Headers, Cookies. **DOES NOT** care about IP, packets, cables. Creates the message.

Layer 4 — Transport Layer (TCP)

"How do I safely deliver it?"

TCP Segmenting & Handshake: Client → SYN, Server → SYN-ACK, Client → ACK. Adds Seq#, Source Port (50000), Dest Port (443).
Guarantees: Order, Delivery, Reliability.

Layer 3 — Network Layer (IP)

"Where should this go?"

IP Addressing:
Source: 192.168.1.5, Destination: 93.184.216.34
Routers read IP, decide path. IP does NOT care if data is lost.

Layer 2 — Data Link Layer

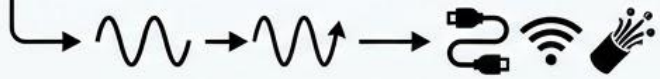
"How do I send it inside my local area?"

Local Delivery: Laptop MAC → Router MAC.
Ethernet Frame. Switches operate here.

Layer 1 — Physical Layer

"How do I physically transmit 0 and 1?"

Physical Transmission: Electrical Signals, WIFI Radio, Light Pulses. Actual bits: 010110...



Processes request, sends response back.

SERVER SIDE (UPWARD F

- Application
- Transport
- Network
- Data Link
- Physical

CONNECT TO REAL PROBLEMS

- 500 Internal Server Error -> Application Layer (Server crash)
- Page loads forever -> TCP Handshake/Routing issue
- WiFi Disconnects -> Physical Layer
- Packets out of order -> TCP fixes it

WHY THIS MATTERS FOR YOU (DEV)

- Learning React, Node, APIs?
- API Slow? Ask: Backend (L7)? Packet Loss (L4)? Routing (L3)? WiFi (L1)?
- Think in layers to blame the right component.

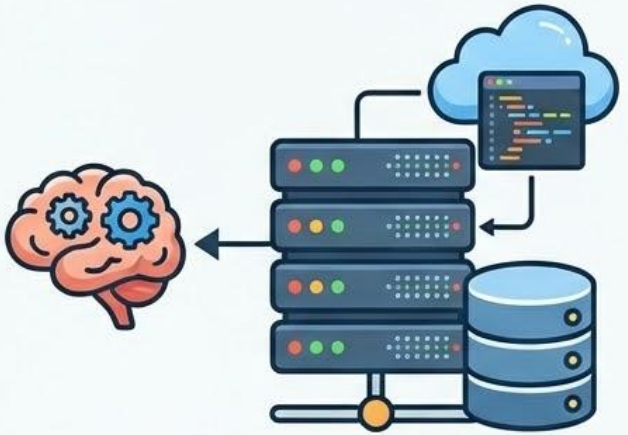
What Is Backend?

Definition: The part of the application that runs on the server and handles logic, data, and security.

Frontend (The Face)

What users see.
UI, buttons, posts.

Backend (The Brain)



What actually makes it work.
Storing posts, login validation, sending data.

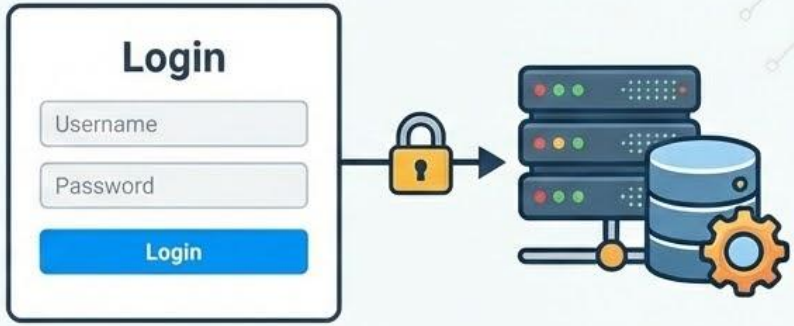
Why Backend Is Required in Full Stack? 🔥

❌ Without Backend (Frontend Only)

You only have static pages.
Anyone can type anything and 'log in'.
That's fake.

No data storage. No verification.

✅ With Backend (The 'Brain' at Work)



- ✅ Authentication (Verifies who you are)
- ✅ Authorization (Controls what you can do)
- ✅ Database operations (Stores data securely)
- ✅ Business logic (Enforces rules)
- ✅ Security (Prevents hacking)
- ✅ Data validation (Checks for correct data)

No backend → No real application.